VeriList

A Blockchain Registry to Stop Fake and Duplicate Product Listings on Online Marketplaces

Project Synopsis Report

Name: Arwindh C N	**Register No.:** cb.sc.u4cse24207	**Course code:** 23CSE335
Name: Raghav V S	**Register No.:** cb.sc.u4cse24243	**Course:** Block Chain and its Applications
Faculty: Dr. Uma J	**Date:** October 2026	**Code:** github.com/ArwindhCN/verilist

1. Abstract

On marketplaces such as Flipkart and Amazon, almost anyone can create a listing that claims to sell a genuine branded product, with made-up specifications. The same product can also show different details on different sites, and duplicate listings clutter the results. **VeriList** is a blockchain registry that fixes this at the source. The brand records its products, their details and its authorized sellers on the blockchain. A marketplace can create a listing only when the blockchain confirms that the product is real and active, its brand is verified, and the seller is authorized. Anyone can verify a listing live, and the answer changes the moment a brand or the admin changes something. The project has four Solidity smart contracts, a React website, 210 automated tests with 100% coverage, a static-analysis scan, a headless-browser test of the website, and a live deployment on the Sepolia test network with the contract code verified on Etherscan.

2. Problem Statement and Motivation

- **Fake listings.** Nobody checks that a seller really may sell the brand's product, or that the specs are true.
- **Inconsistent details.** The same product shows different specs and prices on different sites.
- **Duplicates.** The same product by the same seller is listed again and again.
- **No shared record.** Each marketplace keeps its own private brand list, so a brand must register everywhere and the lists never agree.

Why a blockchain? Brands, marketplaces and the admin do not trust each other, so none of them should own the database. A blockchain gives one shared record that anyone can check without permission, and a history that cannot be quietly rewritten. **Scope:** online listing fraud only. Physical counterfeits (QR codes, NFC, delivery scanning), payments and escrow are out of scope.

3. Objectives

- Make the **brand the single source of truth** for product identity, details and authorized sellers.
- Allow a listing **only** when the product is real and active, its brand is verified and the seller is authorized.
- Show the **same latest details on every marketplace**, while keeping the full history of changes.
- **Block duplicate** active listings for the same product, seller and marketplace.
- Let anyone **verify a listing live**, with a clear reason whenever it is not genuine.
- **Reduce the risk of one admin wallet** with a shared multisig admin and a timelock.

4. Proposed Solution: How VeriList Works

Roles: the Admin verifies brands and registers marketplaces; a Brand registers products and decides which sellers may sell them; a Marketplace creates listings; a Buyer (anyone) verifies listings. Twelve rules (R1 to R12) are enforced in the contracts and each one is covered by named tests. The key ideas are:

- **Brand-scoped product ID:** `keccak256(brand address, SKU)`. Nobody can register a product under another brand's identity.
- **Versioned details:** a change adds a new version (JSON, its hash, a timestamp). Old versions are never changed, and an update with the same hash is rejected.
- **Duplicate key:** `keccak256(product, seller, marketplace)` points to the active listing, so a second one is rejected. Removing a listing frees the key.

- **Live validity:** `verifyListing` is recalculated on every call: listing active, marketplace active, brand verified, product active, seller authorized. If a brand revokes a seller, every listing of that seller turns "not genuine" at once, and turns back when fixed.
- **Pre-check:** the Marketplace page asks `checkListingEligibility` as you type and shows "Can list" or the exact reason before anything is sent.
- **Admin safety:** two-step ownership transfer (offer, then accept), and an optional **M-of-N multisig with a timelock**, where signers can cancel an action during the waiting time.

5. System Architecture

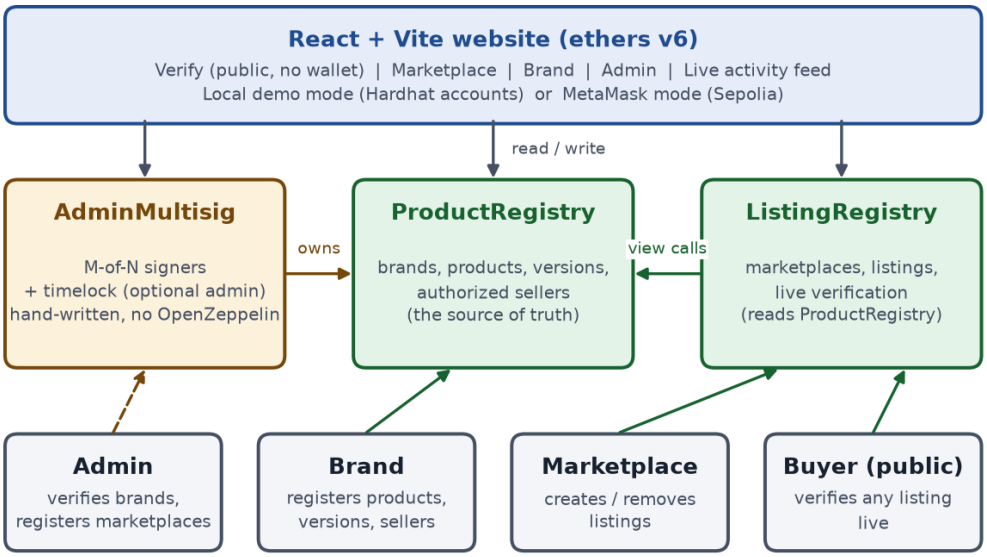


Figure 1: VeriList architecture. Brands write to ProductRegistry; ListingRegistry only reads from it; the admin can be a multisig.

6. Tools and Technologies Integrated

Tool	Purpose	How it is integrated
Solidity 0.8.24	Smart contracts	4 contracts, custom errors, NatSpec on everything, zero compiler warnings
Hardhat 2 + toolbox	Compile, test, local chain, deploy	`npm run compile / test / node / deploy:local`; scripts deploy, seed, e2e-check, verify-all
Mocha + Chai	Unit and integration tests	210 tests; rules tagged [R1] to [R12]; integration story tests
solidity-coverage	Test coverage	100% statements, branches and functions on all four contracts
Slither (Trail of Bits)	Static security analysis	0 High; 5 Medium justified in writing; one false positive removed by a code change
ethers.js v6	Talks to the blockchain	Website and scripts read and write the contracts; revert errors are decoded
React 19 + Vite 8	Website	Verify, Marketplace, Brand, Admin tabs and a live activity feed
MetaMask / Hardhat accounts	Wallets	Local demo mode (account switcher) and MetaMask mode for Sepolia (automated tests used a mock wallet)
Playwright	Browser automation tests	19-check smoke test (including keyboard focus, 375 px width, reduced motion) and a demo replay
Etherscan + hardhat-verify	Public block explorer	`npm run verify:sepolia` published the source of both contracts (verified); the website links every transaction to Etherscan
Git + GitHub (gh CLI)	Version control and sharing	Public repository `ArwindhCN/verilist` with README, VIVA_GUIDE and DECISIONS
Node.js + dotenv	Runtime and settings	Cross-platform scripts; secrets stay in a git-ignored `.env`

7. Implementation Highlights

- **Contracts:** `Ownable` (two-step), `ProductRegistry` (source of truth), `ListingRegistry` (listings and live checks), `AdminMultisig` (shared admin). Hand-written, no OpenZeppelin; no payable functions; no loops in functions that write; every state change emits an event.

- **Scripts:** `deploy` writes the addresses and ABIs for the website; `seed` creates 3 products and 3 genuine listings; `e2e-check` runs a 24-step story; `verify-all` runs compile, tests, coverage, e2e and the website build in order.
- **Website:** a role-aware interface (it explains why an account cannot act), a friendly sentence for every custom error, a live activity feed, a **Name this address** box so a seller can be given a friendly name inside the site (saved in the browser only, nothing on the chain), and a layout that works at 375 px.
- **5-minute demo:** verify a genuine listing; try to list for an unauthorized seller (rejected twice); update the details (both marketplaces show version 2); revoke a seller (listing becomes "not genuine"); re-authorize (genuine again).

8. Testing and Results

Check	Result
Hardhat tests (Mocha/Chai)	**210 passing**
Coverage	**100%** statements, branches, functions and lines (all four contracts)
End-to-end script	**24 / 24** steps; exit code 1 on any failure was verified
Mutation check	5 deliberate bugs put into the multisig and ownership code; **all 5 caught** by the tests
Mutation check (address names)	15 deliberate bugs put into the naming code; 14 caught, the 1 left changes nothing observable
Slither	0 High; 5 Medium (all explained); 4 Low; 2 informational
Browser smoke test (Playwright)	**19 / 19** checks pass; demo replay passes repeatedly
Sepolia deployment	**Live and verified on Etherscan**: both contracts (Solidity 0.8.24, optimizer 200 runs)
Compiler warnings	**0**

Live on the Sepolia test network (October 2026). The source code of both contracts is public and verified on Etherscan, so anyone can read it and call its functions:

What	Address on Sepolia / website	Link
ProductRegistry	`0x8CDe83A53062Dafa7122B2b268A1C916cD123e14`	View verified code
ListingRegistry	`0xEcA6cF8891201C4D83F503c439520Fc9aDFf4C57`	View verified code
Website (needs MetaMask on Sepolia)	https://enchanting-sunshine-b5a619.netlify.app	Open the website

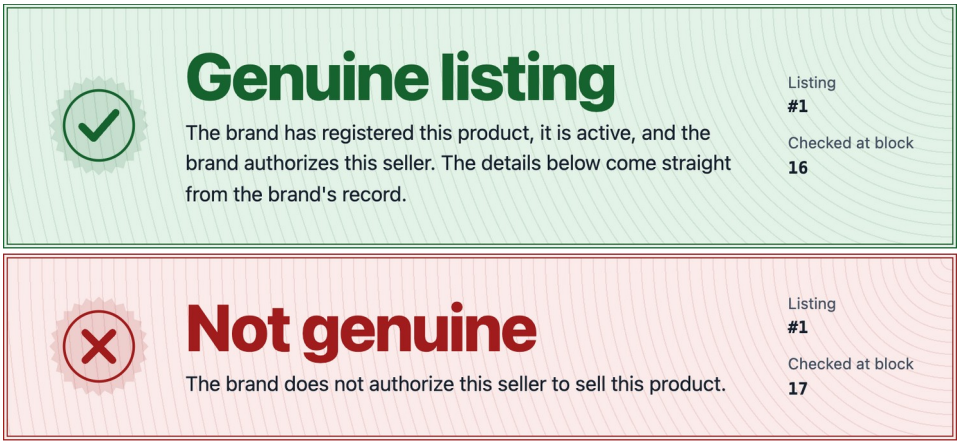


Figure 2: The Verify page. Top: a genuine listing. Bottom: the same listing after the brand revokes the seller. No listing was edited; the answer is calculated live.

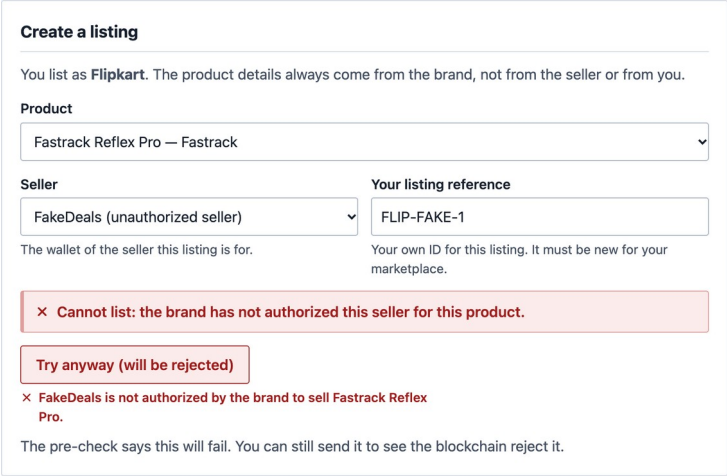


Figure 3: As Flipkart, listing a product for the unauthorized seller FakeDeals. The pre-check refuses it before anything is sent, and the transaction is rejected with a plain sentence. Nothing is written to the chain.

9. Real-World Impact

- **Buyers** can check any listing in seconds, with no account, and see exactly why it is not genuine.
- **Honest brands** register once, control who sells their products, and every site shows the same details.
- **Marketplaces** share one public record instead of many private, disagreeing brand lists.
- **Everyone** gets a permanent audit trail: who changed what, and when.

The value depends on adoption, so a realistic start is a pilot with one marketplace and a few brands.

10. Limitations and Future Scope

Limitations (stated honestly):

- Marketplaces must choose to use it, and the chain proves what the brand **said**, not that it is true.
- Deciding that a wallet really belongs to a brand needs a real-world process; the admin is still a trusted group.
- Data on a public chain is visible to everyone, and physical counterfeits are not detected.
- The contracts are live on Sepolia and verified on Etherscan, but the website has not yet been run end to end with a real MetaMask (automated tests used a mock wallet). The multisig is a small teaching version and is not audited.

Future scope:

- Store details on **IPFS** (hash on chain); **EIP-712** signed seller consent; **Safe** or DAO voting for the admin; real brand verification (for example domain proof); **The Graph** for indexing; a low-cost Layer-2 network; QR codes per unit and escrow.

11. Conclusion

VeriList shows that the brand's own record can act as one shared, public and tamper-evident source of truth for product listings. The contracts, tests, scripts and website work together and were verified end to end. The technology works; the hard part in real life is getting marketplaces and brands to agree to use it. A realistic next step is a pilot with one marketplace and a few brands.

12. References

- Solidity documentation: docs.soliditylang.org | Hardhat documentation: hardhat.org
- ethers.js v6 documentation: docs.ethers.org/v6 | React: react.dev | Vite: vite.dev
- Slither static analyzer (Trail of Bits): github.com/crytic/slither
- EIP-712, Typed structured data signing: eips.ethereum.org/EIPS/eip-712 | Safe multisig: safe.global
- Project source code, README, VIVA_GUIDE and DECISIONS: github.com/ArwindhCN/verilist